

1. Fully-Connected Layer (FC, 완전연결층)

모든 입력값 → 모든 출력값 관계를 모델링 (any output ← any input).

$$s = Wx + b$$

- n=입력수, m=출력수일 때: $x \in \mathbb{R}^{n \times 1}$, $s \in \mathbb{R}^{m \times 1}$, $W \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^{m \times 1}$
- 예: n=1024, m=128 → W는 128×1024

FC Layer의 4가지 문제점

- 파라미터 수 과다: 가중치 多 → 메모리·연산 비용 큼
 - 공간정보 손실: flatten 시 spatial 구조 무시
 - 지역성(Locality) 부재: 가까운 픽셀이 더 연관됨을 못 살림
 - 이동 불변성(Translation Invariance) 부재: 위치만 옮겨도 같은 패턴이 완전히 다르게 인코딩됨
- 해결책: **Convolution Layer**

2. CNN 개요

- 주로 **convolution layer**로 구성, 이미지 같은 **grid 데이터**에 특화
- 전형 구조: [CONV+ReLU → POOL]×N → FLATTEN → FC → SOFTMAX
- Feature Learning**(Conv/Pool) + **Classification**(FC/Softmax)

3. 디지털 이미지 표현

- 픽셀의 grid. 컬러는 **RGB** 3값(0~255)
- Tensor로 취급: $(H \times W \times 3)$ 컬러 / $(H \times W \times 1)$ 흑백

4. Convolution Layer (합성곱층)

- local patch** 입력으로부터 출력 1개 계산
- filter/kernel** 가중치로 지역 특징(local feature) 추출
- 가중치 공유**(weight sharing): 같은 필터를 모든 위치에 적용

예) 입력 3×3 커널 $2 \times 2(0,1,2,3)$ → 출력 2×2
 $0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$ (좌상단), ... = 25, 37, 43

5. Padding (패딩)

- Conv는 층마다 spatial 크기 ↓ ($32 \times 32 \rightarrow 5 \times 5$ conv → $28 \times 28 \rightarrow \dots$)
- 가장자리 픽셀이 덜 쓰임 → **경계 정보 손실**
- 해결**: 입력 주변에 행/열 추가(주로 **zero padding**) → 깊은 망 구성 가능

6. Stride (스트라이드)

- 필터가 입력 위를 움직이는 **step 크기**
- 크기를 빠르게 줄여 **연산·메모리 절감** ($S \uparrow \rightarrow$ 출력 ↓)

★ 7. 출력 크기 공식 (가장 중요!)

$$\text{출력크기} = \lfloor \frac{N + 2P - K}{S} \rfloor + 1$$

N:입력 P:패딩 K:커널 S:스트라이드 $\lfloor \rfloor$:floor

- 홀수 커널**(3×3, 5×5) 사용 → 깨끗한 center
- SAME 패딩**: $P=(K-1)/2$, stride 1 → 출력=입력
- SAME 패딩에서 **S=1** 크기유지 / **S=2** 절반

예) N=32, K=5, P=0, S=1 → $(32-5)/1+1 = 28$

★ 8. 다중 채널 (Multiple Channels)

- 필터 깊이 = 입력 채널 수**. 예: $32 \times 32 \times 3$ 입력 → 필터 $5 \times 5 \times 3$
- 필터 1개 → $5 \cdot 5 \cdot 3 = 75$ 차원 dot product+bias → activation map 1장
- 출력 채널 수 = 필터 개수**
- 필터 형태: $K \times K \times C_{in} \times C_{out}$, bias는 C_{out} 차원 벡터

$$32 \times 32 \times 3 \otimes (5 \times 5 \times 3 \text{ 필터 6개}) \rightarrow 28 \times 28 \times 6$$

※ 채널마다 다른 필터 → 여러 activation map 생성

★ 9. Pooling Layer (풀링층)

- 학습 파라미터 없이** downsampling
- Max Pooling**: patch 내 최대값 → 두드러진 특징 보존
- Average Pooling**: patch 평균 → 부드러운 요약
- 출력 채널 수 = 입력 채널 수** (Conv와 다른 점!)
- 작은 이동(spatial shift)에 **불변성** 부여

$$H' = \lfloor (H-K)/S \rfloor + 1, \quad W' = \lfloor (W-K)/S \rfloor + 1$$

흔한 설정 **K=2, S=2** → 크기 절반. 학습 파라미터 없음.

★★ 10. 주요 CNN 아키텍처 (암기)

모델	핵심 (연도)
AlexNet	ImageNet(ILSVRC'12) 첫 우승. 8층=Conv 5+FC 3. 핵심: Dropout / ReLU / MaxPool
VGGNet	깊이(depth) 강조. 16층=Conv 13+FC 3. 작은 3×3 쌓기가 큰 커널보다 유리
ResNet	Residual connection(shortcut). 152층+ 학습 가능

VGGNet — 3×3 쌓기 이점

- 3×3 conv 3번 = 7×7 1번**과 같은 receptive field
 - 파라미터: $3 \cdot (3 \times 3) = 27C_1C_2 < 7 \times 7 = 49C_1C_2$
 - 같은 RF, 더 적은 파라미터, 더 많은 비선형성

ResNet — 핵심 논리

- plain net을 깊게 쌓으면 **train·test 모두 악화**
 - overfitting 아님 → **최적화 난이도**(optimization difficulty) 때문
- 해결: 입력을 출력에 바로 더하는 shortcut

$$H(x) = F(x) + x$$

- 항등 사상(identity)**을 쉽게 학습 → 깊은 모델 ≥ 얇은 모델
- Basic block**(ResNet-18/34): $3 \times 3 \rightarrow 3 \times 3$
- Bottleneck**(ResNet-50/101/152): $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
- 깊은 ResNet일수록 **에러 낮음** (ResNet-152가 최저)

⚡ 빠른 암기 요약

- Conv 출력채널 = 필터 수 / Pool 출력채널 = 입력 채널 그대로
- Pooling엔 **학습 파라미터 없음**
- SAME 패딩 = $(K-1)/2$, stride 1이면 크기 보존
- 3×3 두 번 ⇒ 5×5 RF, 세 번 ⇒ 7×7 RF
- 층수: AlexNet 8 / VGG16 16 / ResNet 최대 152+